

UNIVERSIDAD DE GUANAJUATO



Programación en Ingenierías

Ejemplos de operadores básicos

Ejemplo 1

- Diseñe un codificador de prioridad de interrupción. Debe poder manejar entradas de 4 dispositivos diferentes, numerados del 1 al 4, y generar la representación binaria del número decimal del dispositivo. Se debe asignar prioridad de manera que el dispositivo con el número más bajo tenga prioridad. Si no se afirma ninguna línea de entrada, la salida debería ser 0.

D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4



D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2	b1	b0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 + D2' * D3$$

$$b1 = D2 * D1' + D1' * D3$$

$$b2 = D1' * D2' * D3' * D4$$

D3 D4	00	01	11	10
D2 D1				
00			1	1
01	1	1	1	1
11	1	1	1	1
10				



D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2	b1	b0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 + D2' * D3$$

$$b1 = D1' * D2 + D1' * D3$$

$$b2 = D1' * D2' * D3' * D4$$

D3 D4	00	01	11	10

D2 D1				
00			1	1
01				
11				
10	1	1	1	1

D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2	b1	b0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 + D2' * D3$$

$$b1 = D2 * D1' + D1' * D3$$

$$b2 = D1' * D2' * D3' * D4$$

D3 D4	00	01	11	10
----- D2 D1				
00		1		
01				
11				
10				



D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2 (4)	b1 (2)	b0 (1)
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 \mid \sim D2 \& D3$$

$$b1 = \sim D1 \& (D2 \mid D3)$$

$$b2 = \sim D1 \& \sim D2 \& \sim D3 \& D4$$

$$\text{Salida} = 4 * b2 + 2 * b1 + b0$$



D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2 (4)	b1 (2)	b0 (1)
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 \mid \sim D2 \& D3$$

$$b1 = \sim D1 \& (D2 \mid D3)$$

$$b2 = \sim D1 \& \sim D2 \& \sim D3 \& D4$$

$$\text{Salida} = 4 * b2 + 2 * b1 + b0$$

$$D4=1, D3=1, D2=0, D1=0$$

$$b0 = 0 \mid 1 \& 1 = 1$$

$$b1 = 1 \& (0 \mid 1) = 1$$

$$b2 = 1 \& 1 \& 0 \& 1 = 0$$

$$\text{Salida} = 4 * 0 + 2 * 1 + 1 = 3$$



D4	D3	D2	D1	Salida
0	0	0	0	0
-	-	-	1	1
-	-	1	0	2
-	1	0	0	3
1	0	0	0	4

Salida	b2 (4)	b1 (2)	b0 (1)
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

$$b0 = D1 \mid \sim D2 \& D3$$

$$b1 = \sim D1 \& (D2 \mid D3)$$

$$b2 = \sim D1 \& \sim D2 \& \sim D3 \& D4$$

$$\text{Salida} = 4 * b2 + 2 * b1 + b0$$

$$D4=1, D3=0, D2=0, D1=1$$

$$b0 = 1 \mid (1 \& 0) = 1$$

$$b1 = 0 \& (0 \mid 0) = 0$$

$$b2 = 0 \& 1 \& 1 \& 1 = 0$$

$$\text{Salida} = 4 * 0 + 2 * 0 + 1 = 1$$



```
#include <stdio.h>

int main(void) {
    int D1, D2, D3, D4, S;
    scanf("%d %d %d %d", &D1, &D2, &D3, &D4);
    S = D1 | (~D2 & D3);
    S |= (((D2 & ~D1) | (~D1 & D3)) << 1);
    S |= (((~D1 & ~D2) & ~D3) & D4) << 2;
    printf("%d\n", S);
    return 0;
}
```

Ejemplo 2

- Considere un robot con sensores de entrada para obstáculo izquierdo (L), obstáculo derecho (R), proximidad a la fuente (P) y batería baja (B). Tiene salidas para mover hacia la derecha (MR), mover hacia la izquierda (ML) y volver a la fuente (S). El robot debe moverse por un área determinada mientras evita los obstáculos y permanece cerca de la fuente. Cuando indica un obstáculo hacia la derecha o hacia la izquierda (R o L son lógicas 1), entonces el robot debe moverse en la dirección opuesta para evitar (establezca MR o ML en la lógica 1 en consecuencia). Cuando ya no esté cerca de la fuente ($P = 0$) o cuando la batería está baja ($B = 0$), entonces debe volver directamente a la fuente (establecer $S = 1$) mientras trata de evitar obstáculos en el camino.

L	R	P	B	ML	MR	S
0	0	1	1	1	1	0
1	0	1	1	0	1	0
0	1	1	1	1	0	0
1	1	1	1	0	0	0/1?
0	0	0/1	0/1	0	0	1
1	0	0/1	0/1	0	1	1
0	1	0/1	0/1	1	0	1
1	1	0/1	0/1	0	0	1



L	R	P	B	ML	MR	S
0	0	1	1	1	1	0
1	0	1	1	0	1	0
0	1	1	1	1	0	0
1	1	1	1	0	0	0/1?
0	0	0/1	0/1	1	1	1
1	0	0/1	0/1	0	1	1
0	1	0/1	0/1	1	0	1
1	1	0/1	0/1	0	0	1

$$S = (P \& B)'$$

$$ML = L'$$

$$MR = R'$$



```
#include <stdio.h>

int main(void) {
    int L, R, P, B, MR, ML, S;
    scanf("%d %d %d %d", &L, &R, &P, &B);
    S=~(P&B);
    ML=~L;
    MR=~R;
    printf("%d\t%d\t%d\n", ML, MR, S);
    return 0;
}
```

Ejemplo 3 (Pokémon Evolution)

You have recently caught N Pokémon and you also have M Pokémon candy bars. You can evolve any of your Pokémon by paying X candy bars. Alternatively, you can sell any of your Pokémon for a price of Y candy bars. You cannot sell an evolved Pokémon.

Compute the maximum number of Pokémon you can evolve.

Standard input

The first line contains 4 integers N M X Y .

Standard output

The output should contain a single integer representing the maximum number of Pokémon you can evolve.

Constraints and notes

$$1 \leq N, M, X, Y \leq 10^9$$

Input	Output	Explanation
5 10 2 1	5	You can evolve all the Pokémon if you use all the available candy.
3 10 4 2	2	You can evolve 2 Pokémon using the initial candy. Selling one Pokémon would result in a total of $10+2=12$ candy bars. This is enough to evolve 3 Pokémon, but you wouldn't have 3 Pokémon left to evolve.
10 3 1 1	6	

Ejemplo 3 (Evolución Pokémon)

Usted recientemente capturó N Pokémon y usted tiene M barras de dulce Pokémon. Usted puede evolucionar cualquiera de sus Pokémon pagando X barras de dulce. Además, usted puede vender cualquiera de Pokémon por un precio de Y barras de dulce. Usted no puede vender un Pokémon evolucionado.

Calcule el máximo número de Pokémon que puede usted evolucionar.

Entrada estándar.

La primer línea contiene 4 enteros N M X Y .

Salida estándar

La salida debe tener un solo entero que representa el número máximo de Pokémon que usted puede evolucionar.

Limitaciones y notas

$$1 \leq N, M, X, Y \leq 10^9$$

Entrada	Salida	Explicación
5 10 2 1	5	Usted puede evolucionar todos los Pokémon con las barras disponibles.
3 10 4 2	2	Usted puede evolucionar 2 Pokémon con los dulces iniciales. Vendiendo un Pokémon usted tendría $10+2=12$ barras de dulce. Esta cantidad es suficiente para evolucionar 3 Pokémon, pero ya no tendría los 3 Pokémon para evolucionar.
10 3 1 1	6	

Ejemplo 3 (Evolución Pokémon)

Entrada	Salida	Explicación
5 10 2 1	5	Usted puede evolucionar todos los Pokémon con las barras disponibles.
3 10 4 2	2	Usted puede evolucionar 2 Pokémon con los dulces iniciales. Vendiendo un Pokémon usted tendría $10+2=12$ barras de dulce. Esta cantidad es suficiente para evolucionar 3 Pokémon, pero ya no tendría los 3 Pokémon para evolucionar.
10 3 1 1	6	

Entradas

N: número de Pokémon iniciales

M: Número de barras de chocolate de Pokémon iniciales

X: Número de barras para evolucionar un Pokémon

Y: Número de barras adquiridas por vender un Pokémon

Variables

K: Número de Pokémon vendidos

S: Número de Pokémon evolucionados

Procedimiento:

Número de barras de chocolates necesarias para evolucionar los Pokémon:

$$(N - K) \times X$$

Número de barras de chocolate que se pueden tener:

$$M + K \times Y$$

Para poder evolucionar los Pokémon necesito tener al menos las barras de chocolate suficientes entonces:

$$(N - K) \times X \leq M + K \times Y$$

Ejemplo 3 (Evolución Pokémon)

Entrada	Salida	Explicación
5 10 2 1	5	Usted puede evolucionar todos los Pokémon con las barras disponibles.
3 10 4 2	2	Usted puede evolucionar 2 Pokémon con los dulces iniciales. Vendiendo un Pokémon usted tendría $10+2=12$ barras de dulce. Esta cantidad es suficiente para evolucionar 3 Pokémon, pero ya no tendría los 3 Pokémon para evolucionar.
10 3 1 1	6	

Procedimiento:

Simplificando:

$$N \times X - K \times X \leq M + K \times Y$$

Separando a K:

$$N \times X - M \leq K \times (X + Y)$$

Despejando a K:

$$\frac{N \times X - M}{X + Y} \leq K$$

En este caso, K debe ser igual o mayor, por lo tanto podemos expresar como:

$$K = \left\lceil \frac{N \times X - M}{X + Y} \right\rceil$$

El número de Pokémon que se puede evolucionar son los que se tenían menos los vendidos:

$$S = N - K$$

Sustituyendo K

$$\begin{aligned}
 S &= N - \left\lceil \frac{N \times X - M}{X + Y} \right\rceil \\
 &= \left\lfloor \frac{N \times X + N \times Y - N \times X + M}{X + Y} \right\rfloor
 \end{aligned}$$

Reduciendo

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor$$

Ejemplo 3 (Evolución Pokémon)

Entrada	Salida	Explicación
5 10 2 1	5	Usted puede evolucionar todos los Pokémon con las barras disponibles.
3 10 4 2	2	Usted puede evolucionar 2 Pokémon con los dulces iniciales. Vendiendo un Pokémon usted tendría 10+2=12 barras de dulce. Esta cantidad es suficiente para evolucionar 3 Pokémon, pero ya no tendría los 3 Pokémon para evolucionar.
10 3 1 1	6	

Entradas

N: número de Pokémon iniciales

M: Número de barras de chocolate de Pokémon iniciales

X: Número de barras para evolucionar un Pokémon

Y: Número de barras adquiridas por vender un Pokémon

Variables

K: Número de Pokémon vendidos

S: Número de Pokémon evolucionados

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = N - \left\lfloor \frac{N \times X - M}{X + Y} \right\rfloor$$

$$= N - K$$

$$N=5, M=10, X=2, Y=1$$

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = \left\lfloor \frac{5 \times 1 + 10}{2 + 1} \right\rfloor = \left\lfloor \frac{5 + 10}{3} \right\rfloor = \left\lfloor \frac{15}{3} \right\rfloor = \lfloor 5 \rfloor = 5$$

$$N=3, M=10, X=4, Y=2$$

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = \left\lfloor \frac{3 \times 2 + 10}{4 + 2} \right\rfloor = \left\lfloor \frac{6 + 10}{6} \right\rfloor = \left\lfloor \frac{16}{6} \right\rfloor = \lfloor 2.6 \rfloor = 2$$

$$N=10, M=3, X=1, Y=1$$

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = \left\lfloor \frac{10 \times 1 + 3}{1 + 1} \right\rfloor = \left\lfloor \frac{10 + 3}{2} \right\rfloor = \left\lfloor \frac{13}{2} \right\rfloor = \lfloor 6.5 \rfloor = 6$$



Ejemplo 3 (Evolución Pokémon)

Entrada	Salida	Explicación
5 10 2 1	5	Usted puede evolucionar todos los Pokémon con las barras disponibles.
3 10 4 2	2	Usted puede evolucionar 2 Pokémon con los dulces iniciales. Vendiendo un Pokémon usted tendría $10+2=12$ barras de dulce. Esta cantidad es suficiente para evolucionar 3 Pokémon, pero ya no tendría los 3 Pokémon para evolucionar.
10 3 1 1	6	

Entradas

N: número de Pokémon iniciales

M: Número de barras de chocolate de Pokémon iniciales

X: Número de barras para evolucionar un Pokémon

Y: Número de barras adquiridas por vender un Pokémon

$N=10, M=20, X=1, Y=1$

$$S = \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = \left\lfloor \frac{10 \times 1 + 20}{1 + 1} \right\rfloor = \left\lfloor \frac{10 + 20}{2} \right\rfloor = \left\lfloor \frac{30}{2} \right\rfloor = \lfloor 15 \rfloor = 15$$

$$\begin{aligned} N \times X &\leq M \\ 10 \times 1 &\leq 20 \rightarrow V(1) \end{aligned}$$

$$\begin{aligned} N \times X &> M \\ 10 \times 1 &> 20 \rightarrow F(0) \end{aligned}$$

Variables

K: Número de Pokémon vendidos

S: Número de Pokémon evolucionados

$$\begin{aligned} S &= \left\lfloor \frac{N \times Y + M}{X + Y} \right\rfloor = N - \left\lfloor \frac{N \times X - M}{X + Y} \right\rfloor \\ &= N - K \end{aligned}$$

$$S = 1 \times N + 0 \times 15 = N = 10$$



```
#include <stdio.h>
```

```
int main(void) {  
    long int N, M, X, Y, K, S;  
    scanf("%ld%ld%ld%ld", &N, &M, &X, &Y);  
    // condicion?verdadera:falsa  
    K=((N*X-M)/(X+Y));  
    S=N-(K>=0?K:0);  
    printf("%ld\n", S);  
    return 0;  
}
```



Red Blue Teams

N children are playing a game. They are split in two teams, R of them are in the red team and $N - R$ are in the blue team.

You also know that exactly K of them will switch teams. Find the minimum and maximum possible number of children in the red team after they all switch teams.

Standard input

The first line contains three integers N , R and K .

Standard Output

Print two integers, the minimum and the maximum possible numbers of children in the read team.

Constraints and notes

$$1 \leq N \leq 10^5$$

$$0 \leq R, K \leq N$$

A child cannot switch side more than once

Equipos rojo y azul

N niños están jugando un juego. Ellos se dividen en dos equipos, R de ellos están en el equipo rojo y $N - R$ están en el equipo azul.

Usted conoce que exactamente K de ellos se cambian de equipo. Busque el número mínimo y máximo de niños en el equipo rojo después que todos ellos hayan cambiado de equipo.

Entrada estándar

La primer línea contiene tres números enteros N , R y K .

Salida estándar

Imprime dos enteros, el mínimo y máximo numero posible de niños en equipo rojo.

Restricciones y notas

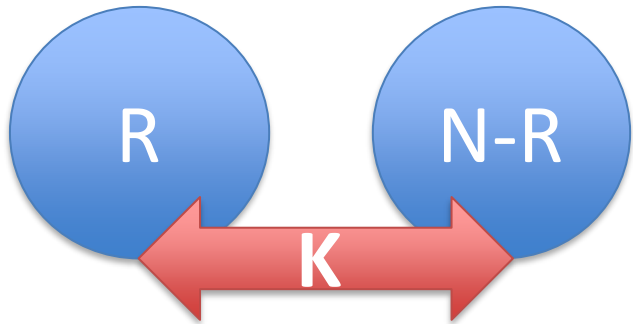
$$1 \leq N \leq 10^5$$

$$0 \leq R, K \leq N$$

Un niño solo puede cambiarse una sola vez



Entrada	Salida	Explicación
6 3 2	1 5	Para el número mínimo, 2 niños pueden cambiarse del equipo rojo al azul. Para el número máximo, 2 niños pueden cambiarse del equipo azul al rojo.
4 3 2	1 3	Para el número mínimo, 2 niños pueden cambiarse del equipo rojo al azul. Para el número máximo, 1 niños puede cambiarse del equipo azul al rojo y 1 niño del rojo al azul. Es importante notar que exactamente K, 2 en este caso, niños deben cambiarse de equipo.
4 3 4	1 1	En este caso todos los niños deben cambiar de equipo, por ello, los 3 niños en el equipo rojo y el niño en el equipo azul se cambian. Después del cambio, solo un niño queda en el equipo rojo.
10 7 2	5 9	



$$R \geq K = \{1, 0\}$$

$$2(R \geq K) = \{2, 0\}$$

$$2(R \geq K) - 1 = \{1, -1\}$$

1. Mínimo número de niños:

1.1. Supongamos que $R \geq K$

$$R_{min} = R - K$$

1.2 Supongamos que $R < K$

$$R_{min} = K - R$$

Uniendo los casos 1.1 y 1.2

$$R_{min} = (R \geq K)(R - K) + (R < K)(K - R)$$

$$R_{min} = (R \geq K)(R - K) + (R < K)(-(R - K))$$

$$R_{min} = \{1, -1\}(R - K)$$

$$R_{min} = (2(R \geq K) - 1)(R - K)$$

2. Máximo número de niños:

2.1 Supongamos que $N - R \geq K$

$$R_{max} = R + K$$

2.2 Supongamos que $N - R < K$

$$R_{max} = R + (N - R) - (K - (N - R))$$

$$R_{max} = 2N - K - R$$

$$R_{max} = 2N - (R + K)$$

Uniendo los casos 2.1 y 2.2

$$R_{max} = (N - R < K)(2N) + (2(N - R \geq K) - 1)(R + K)$$

$$R_{min} = (2(R \geq K) - 1)(R - K)$$

$$R_{max} = (N - R < K)(2N) + (2(N - R \geq K) - 1)(R + K)$$

$$N = 6, R = 3, K = 2$$

$$R_{min} = (2(3 \geq 2) - 1)(3 - 2)$$

$$R_{min} = (2(1) - 1)(1) = (2 - 1) = 1$$

$$R_{max} = (6 - 3 < 2)(2 \cdot 6) + (2(6 - 3 \geq 2) - 1)(3 + 2)$$

$$R_{max} = (0)(12) + (2(1) - 1)(5)$$

$$R_{max} = (2 - 1)(5) = 5$$

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int N, R, K, Rmin, Rmax;
```

```
    scanf("%d %d %d", &N, &R, &K);
```

```
    Rmin = (2*(R>=K)-1)*(R-K);
```

```
    Rmax = ((N-R)<K)*(2*N)+(2*((N-R)>=K)-1)*(R+K);
```

```
    printf("%d %d", Rmin, Rmax);
```

```
    return 0;
```

```
}
```


UNIVERSIDAD DE GUANAJUATO

